otmeshing:
Advanced Mesh Generation for OpenTURNS

OpenTURNS Team

otmeshing – CGAL-powered geometric meshing

June 11, 2026

Why?

Needed for:

- the definition of distributions supported by complex domains **UniformOverMesh**, **TruncatedOverMesh**
- the computation of the CDF of such distributions
- the definition of constrained domains for sampling using eg. **GaussianProcess** or **LOLAVoronoi**

Overview

otmeshing provides 9 classes for mesh generation and manipulation:

- **CloudMesher** – triangulate point clouds
- **ConvexHullMesher** – convex hull surface
- **ConvexDecompositionMesher** – split into convex parts
- **Cylinder** – base mesh $\times$ interval
- **FunctionGraphMesher** – sub/super-graph of f
- **IntersectionMesher** – intersection of meshes
- **MeshDomain2** – signed distance from meshes
- **PolygonMesher** – 2D polygon triangulation
- **UnionMesher** – disjoint mesh union

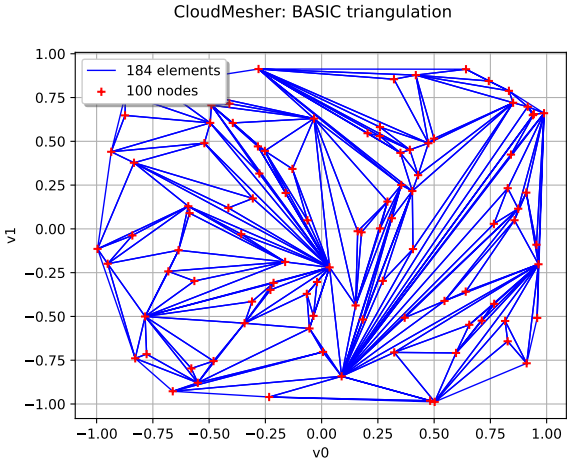
Built on **CGAL**, with optional **Qhull** and **cddlib** support.

CloudMesher – Point Cloud Triangulation

Triangulates a set of points into a volumetric simplicial mesh.

- `BASIC`: fast CGAL triangulation
- `DELAUNAY`: Delaunay triangulation (empty-ball property)
- Handles dimensions $d \geq 1$

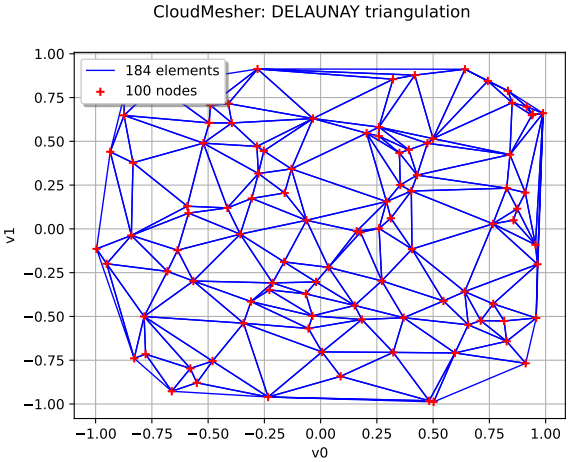
```
mesher = otm.CloudMesher()
mesh = mesher.build(points)
```



CloudMesher – Delaunay Variant

Delaunay triangulation maximises the minimum angle and satisfies the empty circumsphere property.

```
mesher = otm.CloudMesher(
otm.CloudMesher.DELAUNAY)
mesh = mesher.build(points)
```

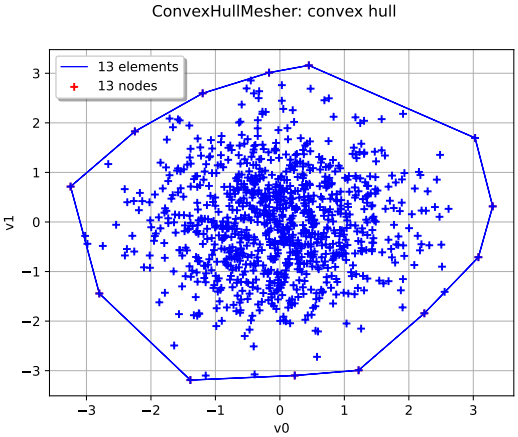


ConvexHullMesher – Convex Hull Surface

Computes the *surface* convex hull (intrinsic dimension $d-1$).

- Uses Qhull (preferred) or CGAL fallback
- Returns a surface mesh
- Supports any dimension

```
mesher = otm.ConvexHullMesher()
hull = mesher.build(points)
```

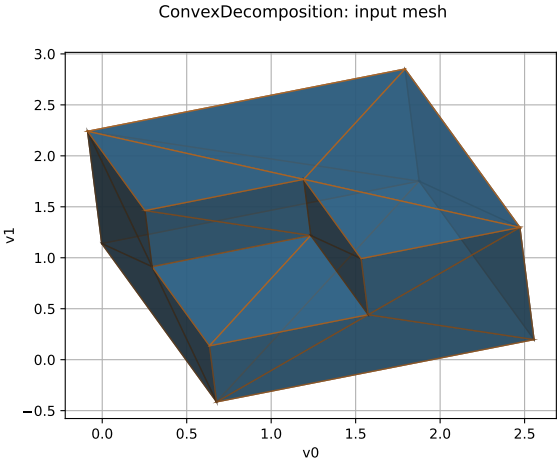


ConvexDecompositionMesher – Mesh Decomposition

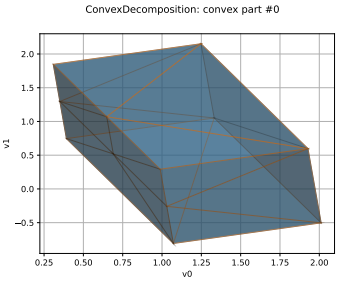
Decomposes a non-convex mesh into convex components.

- 2D: connected components + greedy merging
- 3D surface: CGAL Nef polyhedron decomposition
- 3D volumetric: per-tetra Nef union + decomposition
- Higher dim: per-simplex decomposition

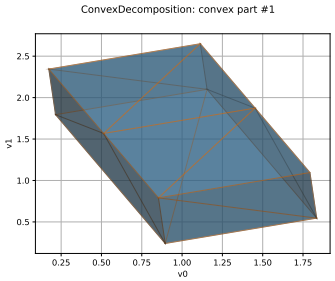
```
mesher = otm.ConvexDecompositionMesher()
parts = mesher.build(mesh)
```



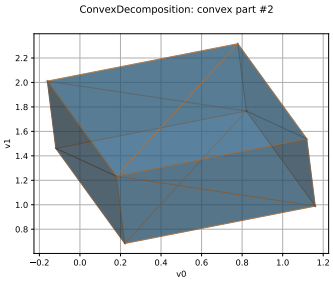
ConvexDecompositionMesher – Convex Parts



Part 0



Part 1



Part 2

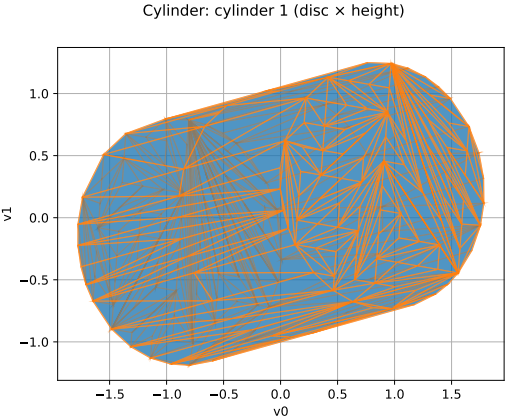
Also provides `IsConvex(mesh)` to test convexity.

Cylinder – Generalized Cylinder

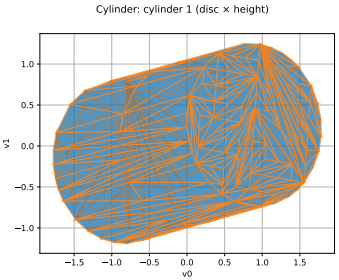
Cartesian product of a *base mesh* and an *extension interval*.

- `base`: any mesh (e.g. disc, polygon)
- `extension`: interval along new dimensions
- `injection`: maps extension dims to output dims
- Supports convexity detection

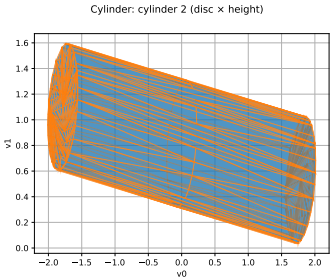
```
cyl = otm.Cylinder(base, interval, inj, disc)
mesh = otm.CloudMesher().build(cyl.
getVertices())
```



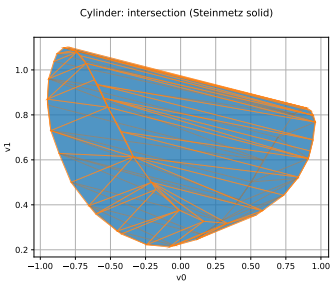
Cylinder – Intersection (Steinmetz Solid)



Cylinder 1 (X-aligned)



Cylinder 2 (Z-aligned)



Intersection

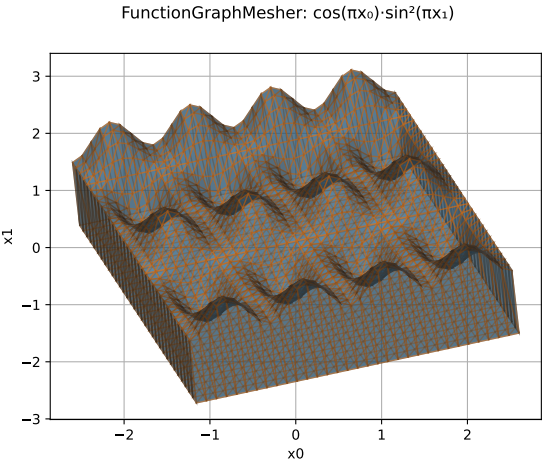
```
inter = otm.IntersectionMesher().buildConvex([m1, m2])
```

FunctionGraphMesher – Function Graph Meshing

Builds a $(n+1)$-D mesh of $\{(x, y) : y \leq f(x)\}$ or $\{(x, y) : y \geq f(x)\}$.

- Input: $f : \mathbb{R}^n \to \mathbb{R}$ (scalar)
- Tensor-product discretisation of input space
- Vertices displaced along the output axis

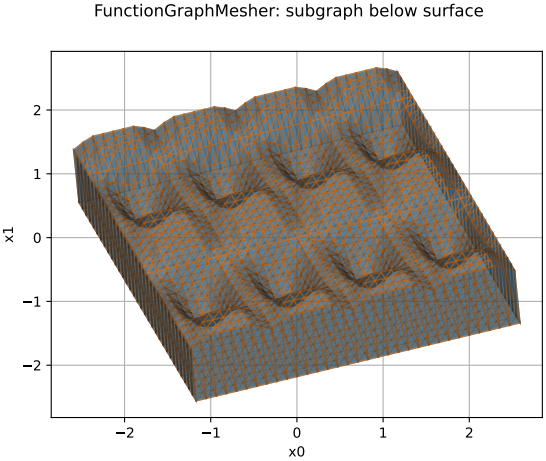
```
mesher = otm.FunctionGraphMesher(bbox,
disc)
mesh = mesher.build(f, idx, ymin, ymax)
```



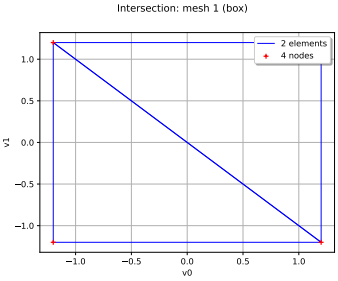
FunctionGraphMesher – Subgraph Example

The same function with the output bound lowered: the mesh is clipped at $y = 0$, creating a subgraph below the surface.

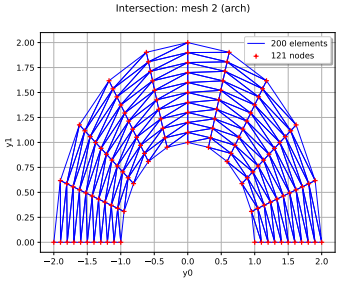
```
mesh = mesher.build(f, 2, -3.5, 0.0)
```



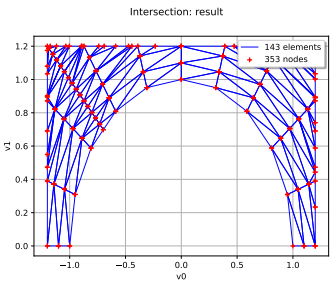
IntersectionMesher – Mesh Intersection



Box $[-1.2, 1.2]^2$



Arch mesh (polar)



Intersection

```
mesher = otm.IntersectionMesher() inter = mesher.build([mesh1, mesh2])
inter = mesher.buildConvex([c1, c2]) (fast convex path)
```

IntersectionMesher – Capabilities

- `build(coll)`: general intersection using convex decomposition + cddlib
- `buildConvex(coll)`: fast path for convex meshes (direct V-to-H conversion)
- `buildCylinder(coll)`: optimised for `Cylinder` objects
- `buildConvexSample(coll)`: returns just the intersection vertices
- Parallelised with **TBB**
- Optional vertex recompression (deduplication)

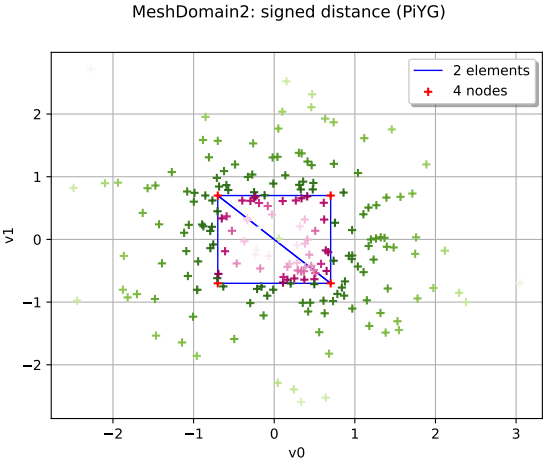
depends on cddlib for polyhedral computations

MeshDomain2 – Signed Distance from Meshes

Adapts any `Mesh` into an OpenTURNS `Domain` with signed distance.

- Negative inside, positive outside
- 2D: CGAL AABB tree on boundary edges
- 3D: CGAL `Side_of_triangle_mesh`
- $d > 3$: quadratic programming on boundary facets

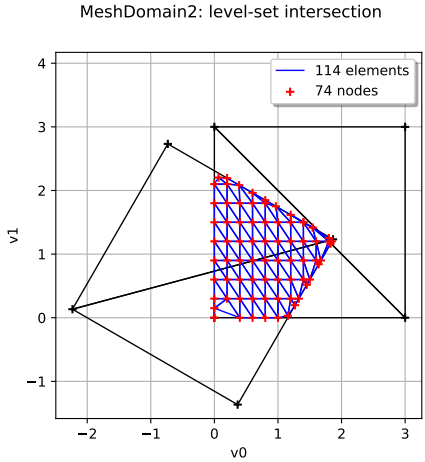
```
domain = otm.MeshDomain2(mesh)
dist = domain.computeDistance(point)
```



MeshDomain2 – Level-Set Intersection

Combine two `MeshDomain2` objects with a max-of-distances level set to compute intersections without cddlib. `def f(X):`

```
return [max(d1(X), d2(X))]
ls = ot.LevelSet(f, ot.LessOrEqual(), 0)
mesh = ot.LevelSetMesher([n]*dim).build(
ls, bbox)
```

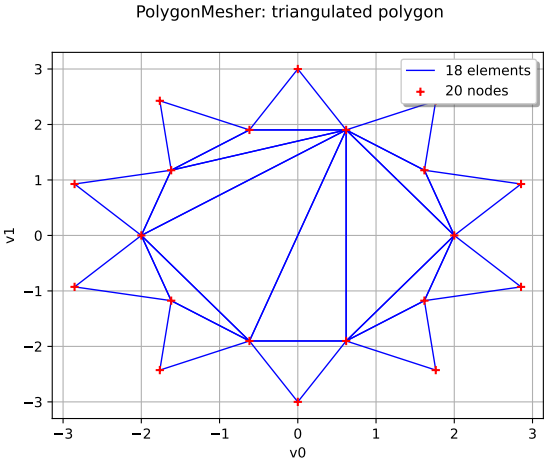


PolygonMesher – 2D Polygon Triangulation

Triangulates a simple (possibly non-convex) 2D polygon.

- Uses CGAL `Polygon_triangulation_decomposit`
- Input: ordered vertices (any ambient dimension ≥ 2)
- Handles non-convex star shapes

```
mesher = otm.PolygonMesher()
mesh = mesher.build(vertices)
```

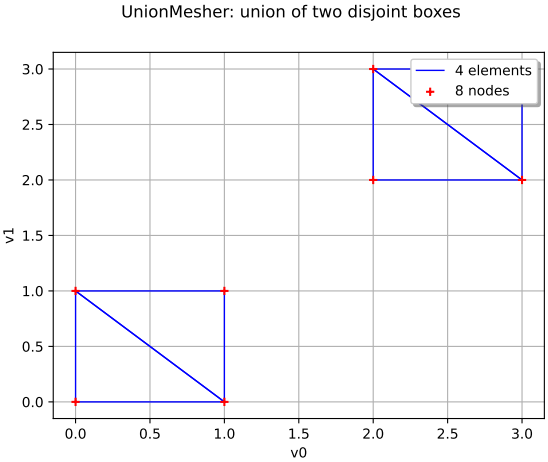


UnionMesher – Disjoint Mesh Union

Concatenates disjoint meshes into a single mesh.

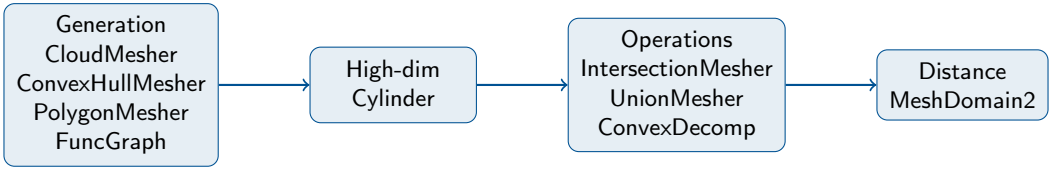
- Vertices concatenated, simplex indices renumbered
- Also provides `CompressMesh` for vertex deduplication
- All meshes must have same dimension

```
mesher = otm.UnionMesher()
union = mesher.build([mesh1, mesh2])
```



Summary

otmeshing provides a complete mesh geometry pipeline:



All classes are serialisable, SWIG-wrapped, and tested for dimensions 1–6.